

The Program in Action

When you run *birthdayparadox.py*, the output will look like this:

```
Birthday Paradox, by Al Sweigart al@inventwithpython.com
--snip--
How many birthdays shall I generate? (Max 100)
> 23

Here are 23 birthdays:
Oct 9, Sep 1, May 28, Jul 29, Feb 17, Jan 8, Aug 18, Feb 19, Dec 1, Jan 22,
May 16, Sep 25, Oct 6, May 6, May 26, Oct 11, Dec 19, Jun 28, Jul 29, Dec 6,
Nov 26, Aug 18, Mar 18

In this simulation, multiple people have a birthday on Jul 29

Generating 23 random birthdays 100,000 times...
Press Enter to begin...
Let's run another 100,000 simulations.
0 simulations run...
10000 simulations run...
--snip--
90000 simulations run...
100000 simulations run.
Out of 100,000 simulations of 23 people, there was a
matching birthday in that group 50955 times. This means
that 23 people have a 50.95 % chance of
having a matching birthday in their group.
That's probably more than you would think!
```

How It Works

Running 100,000 simulations can take a while, which is why lines 95 and 96 report that another 10,000 simulations have finished. This feedback can assure the user that the program hasn't frozen. Notice that some of the integers, like 10_000 on line 95 and 100_000 on lines 93 and 103, have underscores. These underscores have no special meaning, but Python allows them so that programmers can make integer values easier to read. In other words, it's easier to read "one hundred thousand" from 100_000 than from 100000.

```
1. """Birthday Paradox Simulation, by Al Sweigart al@inventwithpython.com
2. Explore the surprising probabilities of the "Birthday Paradox".
3. More info at https://en.wikipedia.org/wiki/Birthday\_problem
4. View this code at https://nostarch.com/big-book-small-python-projects
5. Tags: short, math, simulation"""
6.
7. import datetime, random
8.
9.
10. def getBirthdays(numberOfBirthdays):
11.     """Returns a list of number random date objects for birthdays."""
12.     birthdays = []
```